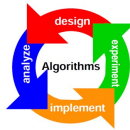


Estrategias algorítmicas

José A. Rodríguez Melquiades

31 de mayo de 2026

Algoritmos recursivos



- 1 Planificación de evacuación con algoritmo recursivo
- 2 Detectar y visualizar múltiples subsecuencias en una cadena de ADN
- 3 Segmentar imágenes usando división recursiva basada en varianza

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.
- **Problema:** Dado el plano de un edificio (expresado como una matriz), se desea hallar todas las posibles rutas para evacuar ante una posible emergencia, empezando desde una posición inicial (persona atrapada) hasta una salida (E), evitando obstáculos (O).

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.
- **Problema:** Dado el plano de un edificio (expresado como una matriz), se desea hallar todas las posibles rutas para evacuar ante una posible emergencia, empezando desde una posición inicial (persona atrapada) hasta una salida (E), evitando obstáculos (O).
- **Información:**

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.
- **Problema:** Dado el plano de un edificio (expresado como una matriz), se desea hallar todas las posibles rutas para evacuar ante una posible emergencia, empezando desde una posición inicial (persona atrapada) hasta una salida (E), evitando obstáculos (O).
- **Información:**
 - ▶ Definir MATRIZ plano como una lista de caracteres.

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.
- **Problema:** Dado el plano de un edificio (expresado como una matriz), se desea hallar todas las posibles rutas para evacuar ante una posible emergencia, empezando desde una posición inicial (persona atrapada) hasta una salida (E), evitando obstáculos (O).
- **Información:**
 - ▶ Definir MATRIZ plano como una lista de caracteres.
 - ▶ Definir ENTERO filas := número del filas de plano.

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.
- **Problema:** Dado el plano de un edificio (expresado como una matriz), se desea hallar todas las posibles rutas para evacuar ante una posible emergencia, empezando desde una posición inicial (persona atrapada) hasta una salida (E), evitando obstáculos (O).
- **Información:**
 - ▶ Definir MATRIZ plano como una lista de caracteres.
 - ▶ Definir ENTERO filas := número del filas de plano.
 - ▶ Definir ENTERO columnas:= número del columnas de plano.

Planificación de evacuación con algoritmo recursivo

- Se desea planificar una evacuación, en una red de edificios o calles, a través rutas de escape.
- **Problema:** Dado el plano de un edificio (expresado como una matriz), se desea hallar todas las posibles rutas para evacuar ante una posible emergencia, empezando desde una posición inicial (persona atrapada) hasta una salida (E), evitando obstáculos (O).
- **Información:**
 - ▶ Definir MATRIZ plano como una lista de caracteres.
 - ▶ Definir ENTERO filas := número del filas de plano.
 - ▶ Definir ENTERO columnas:= número del columnas de plano.
 - ▶ Definir LISTA rutas:= lista vacía.

● Seudocódigo:

```
1: procedure BUSCARRUTAS(x, y, camino)
2:   if x o y están fuera de los límites del plano then
3:     Retornar sin éxito
4:   end if
5:   if plano[x][y] es O (pared) o (x, y) ya está en camino then
6:     Retornar
7:   end if
8:   if plano[x][y] es E then
9:     Agregar camino + (x, y) a rutas
10:    Retornar
11:  end if
12:  Agregar (x, y) a camino
13:  BuscarRutas(x + 1, y, copia de camino)
14:  BuscarRutas(x - 1, y, copia de camino)
15:  BuscarRutas(x, y + 1, copia de camino)
16:  BuscarRutas(x, y - 1, copia de camino)
17: end procedure
```

▷ E significa una salida

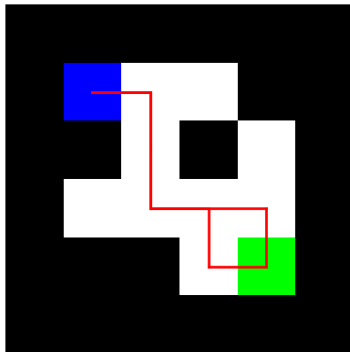
```

1: for i:= 0 : filas - 1 do                                ▷ Buscar coordenadas de inicio S
2:   for j:= 0 : columnas - 1 do
3:     if plano[i][j] es S then
4:       Llamar BuscarRutas(i, j, lista vacía)
5:     end if
6:   end for
7: end for
8: Crear matriz de ceros(imagen) de igual tamaño que el plano    ▷ Visualización
9: for i:= 0 : filas - 1 do
10:  for j:= 0 : columnas - 1 do
11:    if plano[i][j] es O then                                ▷ Obstáculo
12:      imagen[i][j] := color negro
13:    end if
14:    if plano[i][j] es . then ▷ espacio libre para caminar y formar rutas de escape
15:      imagen[i][j] := color blanco
16:    end if
17:    if plano[i][j] es S then                                ▷ Posición inicial de la persona atrapada
18:      imagen[i][j] := color azul
19:    end if
20:    if plano[i][j] es E then                                ▷ Salida
21:      imagen[i][j] := color verde
22:    end if
23:  end for
24: end for

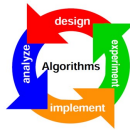
```

- 1: **for** cada ruta en rutas **do** ▷ Dibujar rutas en rojo
- 2: **for** Para cada par consecutivo de puntos $(x1, y1)$ y $(x2, y2)$ en ruta **do**
- 3: Dibujar línea roja entre $(x1, y1)$ y $(x2, y2)$ ▷ Recorrido para escape
- 4: **end for**
- 5: **end for**
- 6: Mostrar número de rutas encontrada

Rutas de evacuación encontradas: 2



Algoritmos recursivos



- 1 Planificación de evacuación con algoritmo recursivo
- 2 Detectar y visualizar múltiples subsecuencias en una cadena de ADN
- 3 Segmentar imágenes usando división recursiva basada en varianza

Detectar y visualizar múltiples subsecuencias en una cadena de ADN

- **Problema:** Dado un fragmento de ADN (una cadena "ACGTGTCAGT"). Se desea detectar si una subsecuencia objetivo ("GTC") está presente siguiendo el orden, pero no necesariamente de forma contigua.

Detectar y visualizar múltiples subsecuencias en una cadena de ADN

- **Problema:** Dado un fragmento de ADN (una cadena "ACGTGTCAGT"). Se desea detectar si una subsecuencia objetivo ("GTC") está presente siguiendo el orden, pero no necesariamente de forma contigua.
- **Importancia:** Es útil para detectar información genética importante en bioinformática.

● Seudocódigo:

```
1: function HALLARSUBSECUENCIA(adn, subsecuencia, i, j, indices)
2:   if j == longitud(subsecuencia) then
3:     Retornar (True, indices)
4:   end if
5:   if i == longitud(adn) then
6:     Retornar (False, indices)
7:   end if
8:   if adn[i] == subsecuencia[j] then
9:     Agregar i a indices
10:    Retornar HallarSubsecuencia(adn, subsecuencia, i+1, j+1, indices)
11:  else
12:    Retornar HallarSubsecuencia(adn, subsecuencia, i+1, j, indices)
13:  end if
14: end function
```

● Seudocódigo: (cont.)

- 1: Definir `adn` como una cadena de caracteres
- 2: Definir `ListaDeSubsecuencias` como una lista de cadenas
- 3: Definir `ListaDeResultados` como una lista vacía
- 4: **for** cada subsecuencia en `ListaDeSubsecuencias` **do**
- 5: Llamar `HallarSubsecuencia`(`adn`, subsecuencia, 0, 0, lista vacía)
- 6: **if** se encontró coincidencia **then**
- 7: Agregar (subsecuencia, `ListaDeIndices`) a `ListaDeResultados`
- 8: **end if**
- 9: **end for**
- 10: Inicializar ventana de visualización con tamaño proporcional a la longitud del ADN
- 11: **for** cada letra en `adn` con su índice `i` **do**
- 12: Mostrar letra en posición horizontal `i` sobre la figura
- 13: **end for**
- 14: **for** cada (subsecuencia, índices) en `ListaDeResultados` **do**
- 15: Asignar un color único para esta subsecuencia
- 16: **for** cada índice en índices **do**
- 17: Dibujar un rectángulo en la posición correspondiente
- 18: **end for**
- 19: **end for**
- 20: Mostrar leyenda con nombre de cada subsecuencia y su color
- 21: Mostrar título y renderizar figura final

- Ejemplo:

- Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC

- Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]

- Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]
- ▶ Resultados de coincidencias:

- Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]
- ▶ Resultados de coincidencias:
 - ★ Subsecuencia: GTC encontrada en posiciones: [2, 3, 6] -> G-T-C

● Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]
- ▶ Resultados de coincidencias:
 - ★ Subsecuencia: GTC encontrada en posiciones: [2, 3, 6] -> G-T-C
 - ★ Subsecuencia: ACT encontrada en posiciones: [0, 1, 3] -> A-C-T

● Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]
- ▶ Resultados de coincidencias:
 - ★ Subsecuencia: GTC encontrada en posiciones: [2, 3, 6] -> G-T-C
 - ★ Subsecuencia: ACT encontrada en posiciones: [0, 1, 3] -> A-C-T
 - ★ Subsecuencia: TAG encontrada en posiciones: [3, 7, 8] -> T-A-G

● Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]
- ▶ Resultados de coincidencias:
 - ★ Subsecuencia: GTC encontrada en posiciones: [2, 3, 6] -> G-T-C
 - ★ Subsecuencia: ACT encontrada en posiciones: [0, 1, 3] -> A-C-T
 - ★ Subsecuencia: TAG encontrada en posiciones: [3, 7, 8] -> T-A-G
 - ★ Subsecuencia: AGT encontrada en posiciones: [0, 2, 3] -> A-G-T

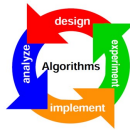
● Ejemplo:

- ▶ adn = ACGTGTCAGTACCTAGCTGATC
- ▶ subsecuencias = [GTC, ACT, TAG, AGT, GAT]
- ▶ Resultados de coincidencias:
 - ★ Subsecuencia: GTC encontrada en posiciones: [2, 3, 6] -> G-T-C
 - ★ Subsecuencia: ACT encontrada en posiciones: [0, 1, 3] -> A-C-T
 - ★ Subsecuencia: TAG encontrada en posiciones: [3, 7, 8] -> T-A-G
 - ★ Subsecuencia: AGT encontrada en posiciones: [0, 2, 3] -> A-G-T
 - ★ Subsecuencia: GAT encontrada en posiciones: [2, 7, 9] -> G-A-T

Coincidencias de subsecuencias en la cadena de ADN



Algoritmos recursivos



- 1 Planificación de evacuación con algoritmo recursivo
- 2 Detectar y visualizar múltiples subsecuencias en una cadena de ADN
- 3 Segmentar imágenes usando división recursiva basada en varianza

Segmentar imágenes usando división recursiva basada en varianza

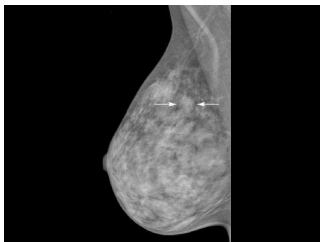
- **Segmentación:** Útil para detectar tumores en mamografías, procesamiento médico o visión por computadora en general.

Segmentar imágenes usando división recursiva basada en varianza

- **Segmentación:** Útil para detectar tumores en mamografías, procesamiento médico o visión por computadora en general.
- **Segmentación recursiva de imágenes por homogeneidad:**

Segmentar imágenes usando división recursiva basada en varianza

- **Segmentación:** Útil para detectar tumores en mamografías, procesamiento médico o visión por computadora en general.
- **Segmentación recursiva de imágenes por homogeneidad:**
 - ▶ **Idea:** Dividir recursivamente una imagen en regiones más pequeñas hasta que cada una sea suficientemente homogénea, medida con la varianza de los valores de píxeles. Este principio se usa en aplicaciones médicas, como detectar regiones sospechosas en una mamografía.



- Aplicaciones:

- Aplicaciones:

- ▶ Detectar tumores en mamografías: Detectar regiones sospechosas.

- Aplicaciones:

- ▶ Detectar tumores en mamografías: Detectar regiones sospechosas.
- ▶ Compresión de imágenes: Dividir en regiones homogéneas y codificar.

- Aplicaciones:

- ▶ Detectar tumores en mamografías: Detectar regiones sospechosas.
- ▶ Compresión de imágenes: Dividir en regiones homogéneas y codificar.
- ▶ Análisis satelital: Dividir terreno en regiones homogéneas.

- **Aplicaciones:**

- ▶ **Detectar tumores en mamografías:** Detectar regiones sospechosas.
- ▶ **Compresión de imágenes:** Dividir en regiones homogéneas y codificar.
- ▶ **Análisis satelital:** Dividir terreno en regiones homogéneas.
- ▶ **Segmentación automática en visión artificial.**

- Aplicaciones:

- ▶ Detectar tumores en mamografías: Detectar regiones sospechosas.
- ▶ Compresión de imágenes: Dividir en regiones homogéneas y codificar.
- ▶ Análisis satelital: Dividir terreno en regiones homogéneas.
- ▶ Segmentación automática en visión artificial.

- Seudocódigo:

- **Aplicaciones:**

- ▶ **Detectar tumores en mamografías:** Detectar regiones sospechosas.
- ▶ **Compresión de imágenes:** Dividir en regiones homogéneas y codificar.
- ▶ **Análisis satelital:** Dividir terreno en regiones homogéneas.
- ▶ Segmentación automática en visión artificial.

- **Seudocódigo:**

- ▶ Segmenta la imagen en regiones cuadrantes recursivamente.

- **Aplicaciones:**

- ▶ **Detectar tumores en mamografías:** Detectar regiones sospechosas.
- ▶ **Compresión de imágenes:** Dividir en regiones homogéneas y codificar.
- ▶ **Análisis satelital:** Dividir terreno en regiones homogéneas.
- ▶ Segmentación automática en visión artificial.

- **Seudocódigo:**

- ▶ Segmenta la imagen en regiones cuadrantes recursivamente.
- ▶ Marca en rojo todas las regiones homogéneas (baja varianza).

- **Aplicaciones:**

- ▶ **Detectar tumores en mamografías:** Detectar regiones sospechosas.
- ▶ **Compresión de imágenes:** Dividir en regiones homogéneas y codificar.
- ▶ **Análisis satelital:** Dividir terreno en regiones homogéneas.
- ▶ Segmentación automática en visión artificial.

- **Seudocódigo:**

- ▶ Segmenta la imagen en regiones cuadrantes recursivamente.
- ▶ Marca en rojo todas las regiones homogéneas (baja varianza).
- ▶ Marca en amarillo las regiones con alta varianza interna $\Rightarrow$ posibles lesiones sospechosas.

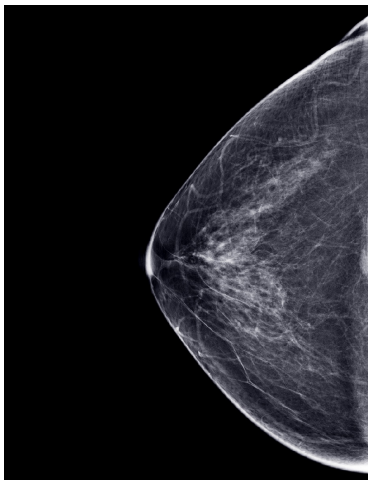
- Seudocódigo: (cont.)

- 1: Convertir imagen a escala de grises
- 2: Normalizar imagen a tipo flotante
- 3: Inicializar lista regiones = []
- 4: Inicializar lista sospechosas = []
- 5: SegmentarDetectar(imagen, x=0, y=0, w=ancho, h=alto, thresholdVar, umbralSospechosa, minSize, regiones, sospechosas)
- 6: Mostrar imagen ▷ rojo regiones en "regiones" y amarillo regiones "sospechosas"

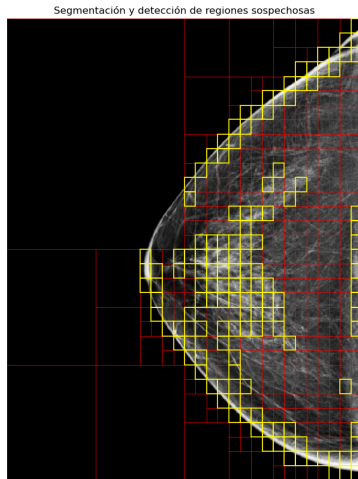
● Seudocódigo: (cont.)

```
1: function SEGMENTARDETECTAR(img, x, y, w, h, thresholdVar,  
   umbralSospechosa, minSize, regiones, sospechosas)  
2:   subImg:= Región de img definida por (x, y, w, h)  
3:   varianza:= Varianza(subImg)  
4:   if ( $w \leq \text{minSize} \parallel h \leq \text{minSize} \parallel \text{varianza} < \text{thresholdVar}$ ) then  
5:     Agregar (x, y, w, h) a las regiones  
6:     if varianza  $\geq$  umbralSospechosa then  
7:       Agregar (x, y, w, h, varianza) a sospechosas  
8:     end if  
9:   else  
10:    mitadw := w // 2  
11:    mitaddh := h // 2  
12:    SegmentarDetectar(img, x, y, mitadw, mitaddh, ...)  
13:    SegmentarDetectar(img, x + mitadw, y, w - mitadw, mitaddh, ...)  
14:    SegmentarDetectar(img, x, y + mitaddh, mitadw, h - mitaddh, ...)  
15:    SegmentarDetectar(img, x + mitadw, y + mitaddh, w - mitadw, h -  
    mitaddh, ...)  
16:  end if  
17:  Retornar regiones, sospechosas  
18: end function
```

- Ejemplo:



(a) Figura de seno inicial.



(b) Figura de seno procesado.

Figura 1: Segmentación y detección de tumor.